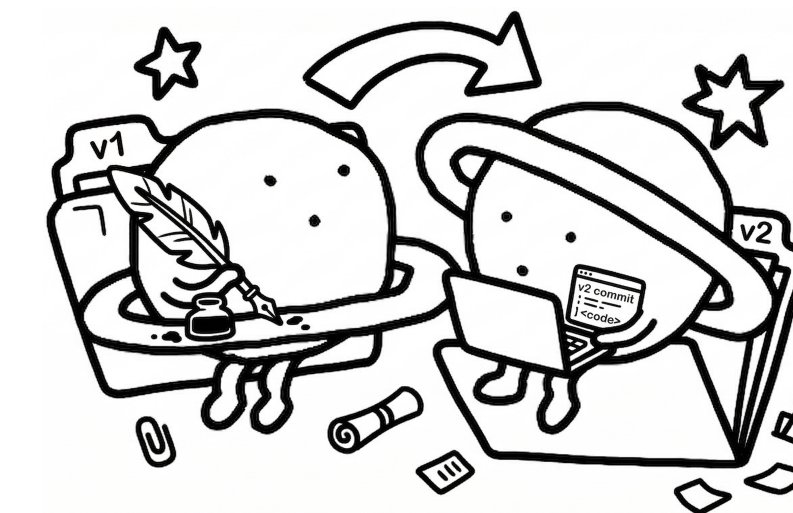
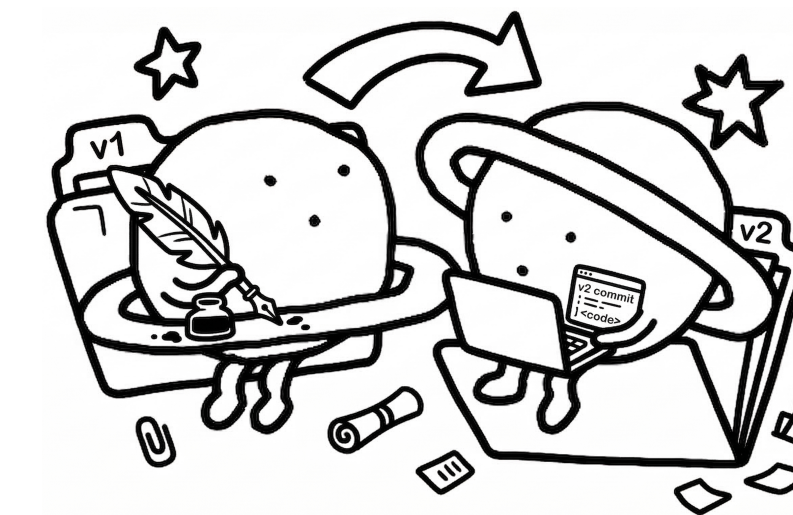




우아한 Null 처리

테크살롱 17기 쿠다

순서

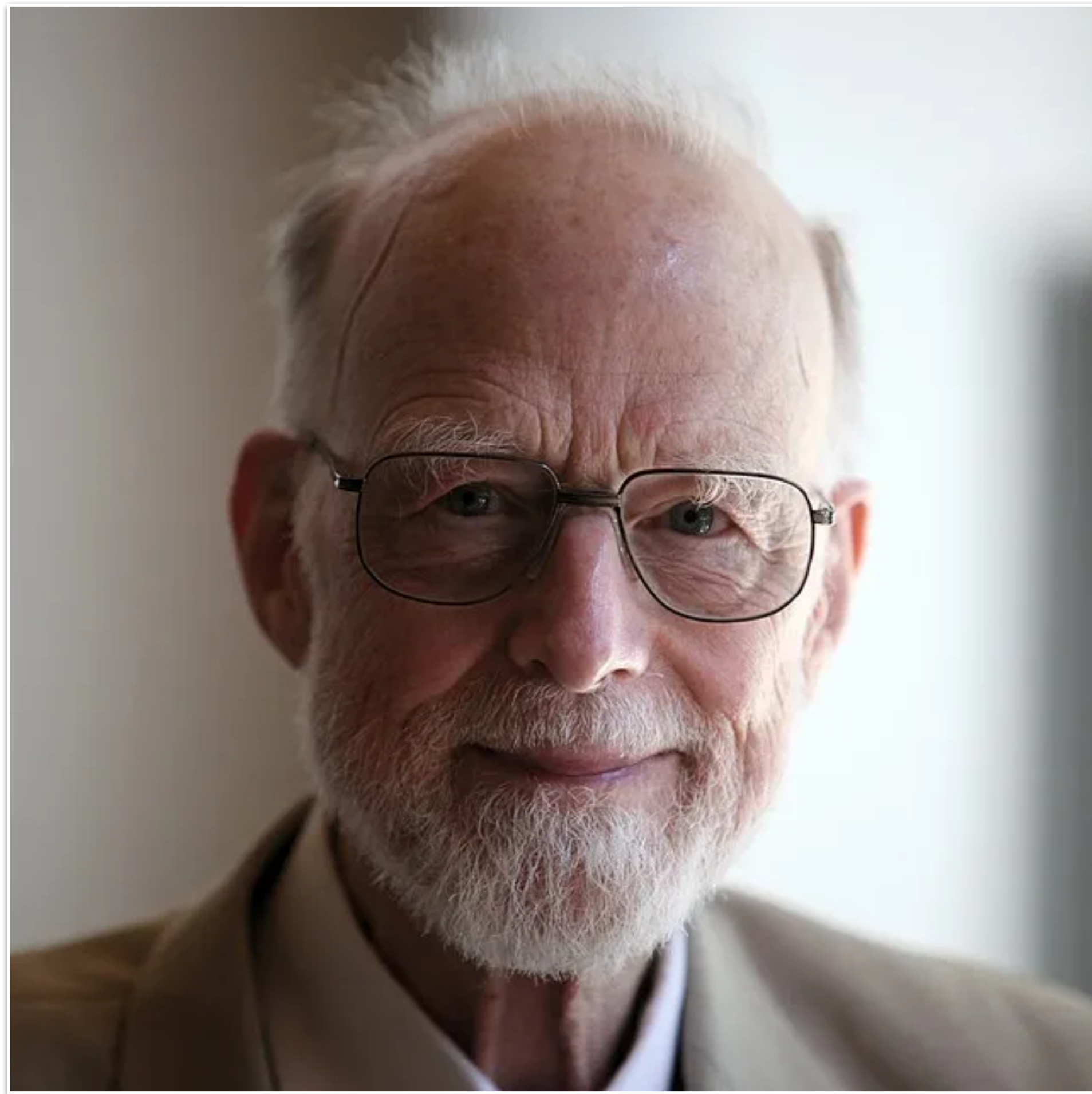
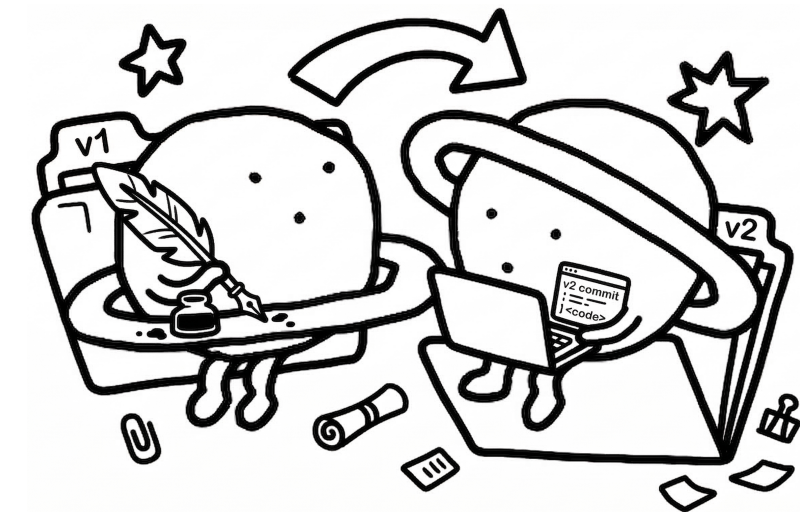


1. NPE

2. Optional

3. Null Object Pattern

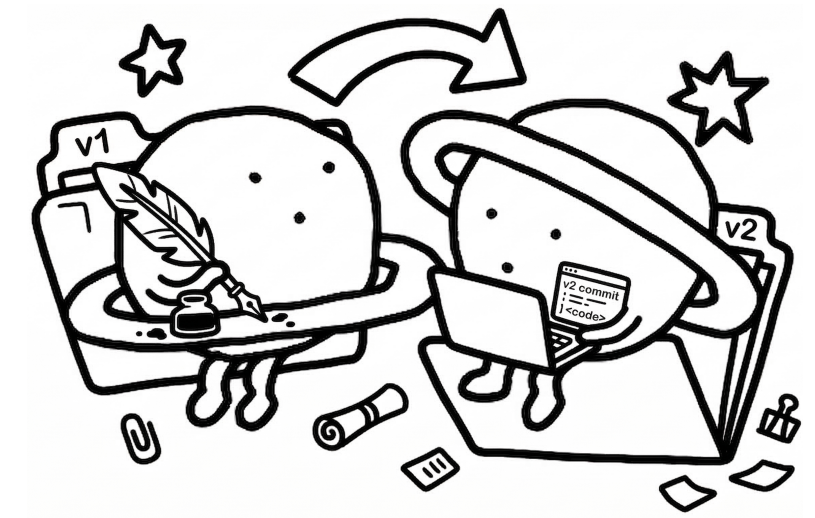
개발자를 괴롭히는 예외 1위 NPE



null은 10억 달러짜리 실수

- 토니 호어

가장 고전적인 방법



if(obj != null)

```
Payment payment = findPayment();

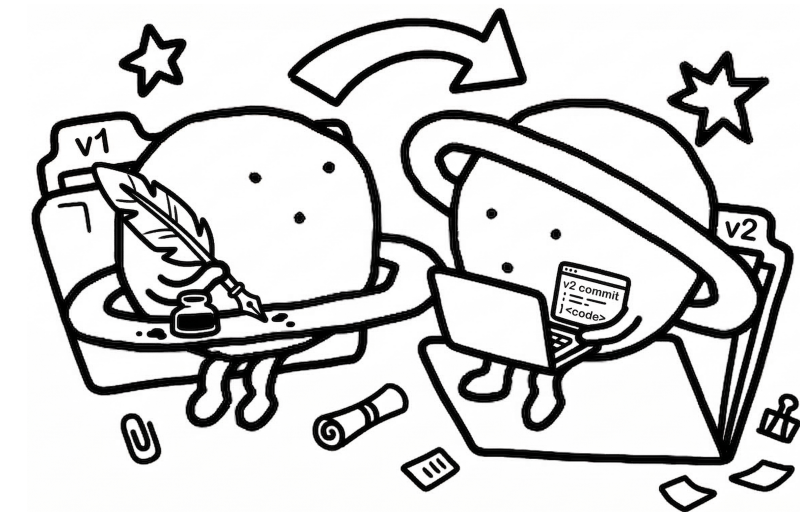
if (payment != null) {
    payment.pay();
}
```

› 직관적

› 코드 반복

› 개발자 실수 시 NPE 발생

Java8 Optional의 등장

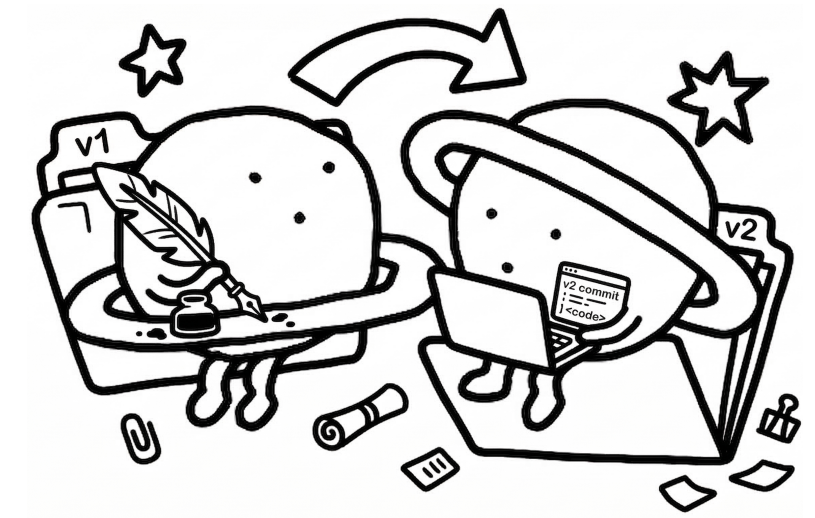


› 명시적으로 안전한 null 처리 유도

› '존재할 수도 있지만, 안 할 수도 있는 객체'

› null이 될 수 있는 객체를 안전하게 감싸는 '레퍼 클래스'이자 NPE를 방지하는 상자

Optional 내부 구현 코드



```
public final class Optional<T> {  
  
    private static final Optional<?> EMPTY = new Optional<>();  
  
    private final T value;  
  
    private Optional(T value) {  
        this.value = Objects.requireNonNull(value);  
    }  
    ...  
    public T get() {  
        if (value == null) {  
            throw new NoSuchElementException("No value present");  
        }  
        return value;  
    }  
}
```

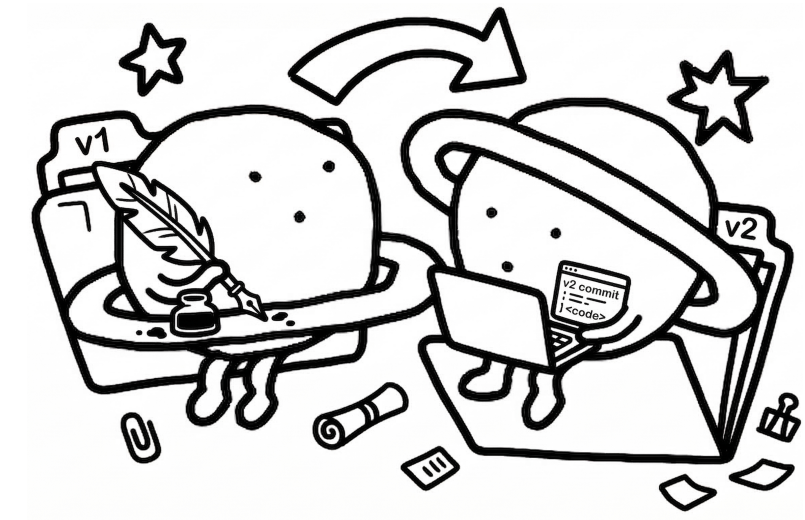
› 캡테기 객체

› 직접 생성 불가

› 불변

› 비용의 존재

Optional의 장점



```
Member member = findById(id);

// 개발자가 null 체크를 깜빡하면
return member.getName(); // NullPointerException 발생 가능
```

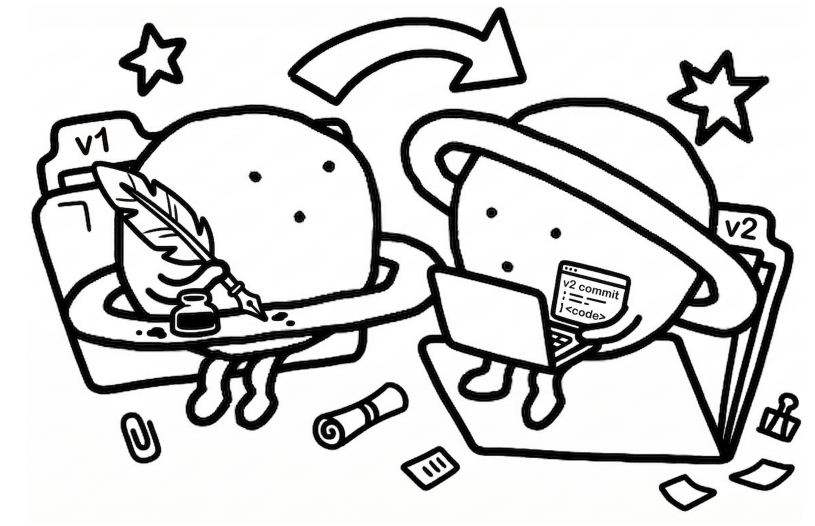
```
Optional<Member> member = findById(id);

// 반드시 값을 꺼내는 방식에 대해 고민해야 함
return member
    .map(Member::getName)
    .orElse("이름 없음");
```

명시적인 Null 발생 알림

철저한 Null 처리 강제화

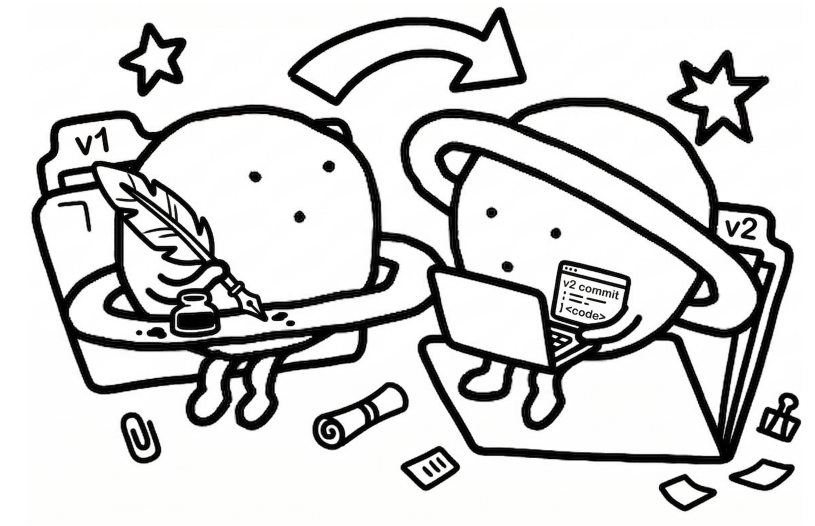
Optional의 단점



```
optional.get()
```

모든 Null 문제를 해결하지 못함

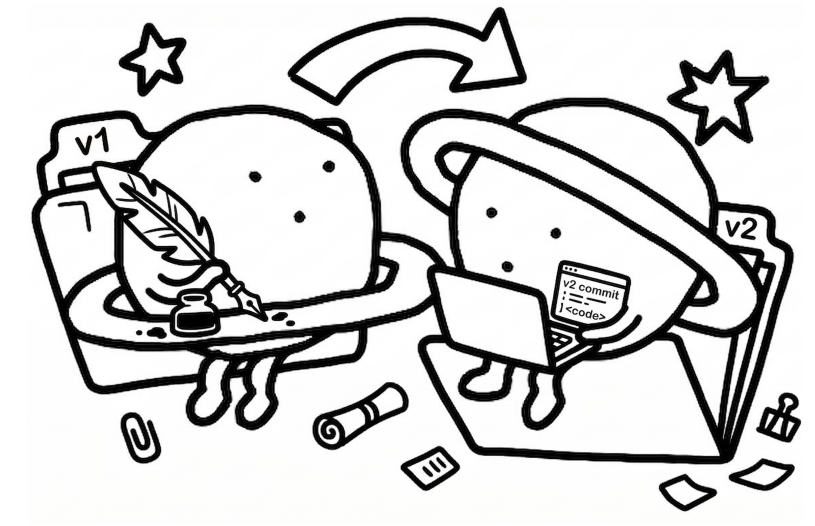
Optional의 단점



```
Optional.ofNullable(user)
    .map(User::getName)
    .map(String::toUpperCase)
    .orElse("UNKNOWN");
```

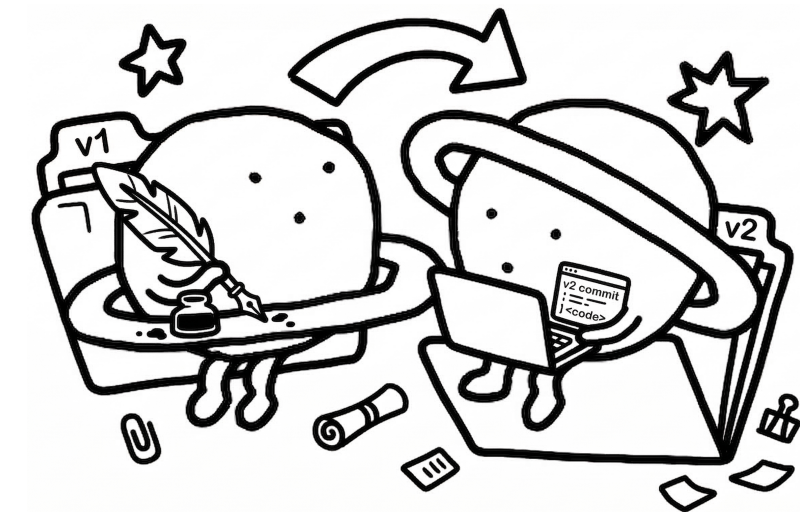
코드가 복잡해짐

Optional의 단점



```
return Optional.of(member);
```

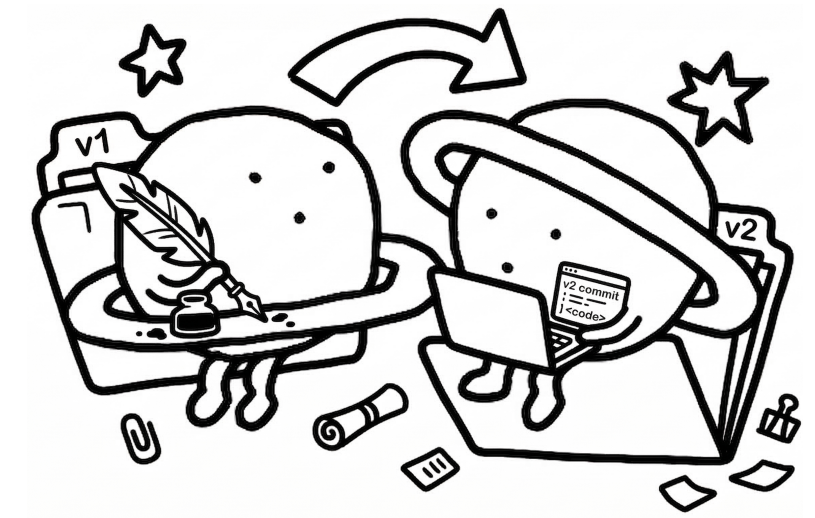
추가 메모리 사용과 GC 비용 발생



Null Object Pattern

“Null 을 반환하는 대신, ‘아무것도 하지 않는’ 진짜 객체를 반환하자”

Null Object Pattern?



면접관

그럼 처음에 "빈칸도 상태를 가진다"고 했던 말이랑 다르잖아.

그냥 Null Object 패턴이라고 처음부터 말했어야 하는 거 아니야?

오후 02:49

그렇게 볼 수 있을 것 같습니다. 현재 빈칸에 대해 따로 상태를 가지지 않기 때문에 null object 쪽이 더 유효할 것 같습니다

오후 02:50

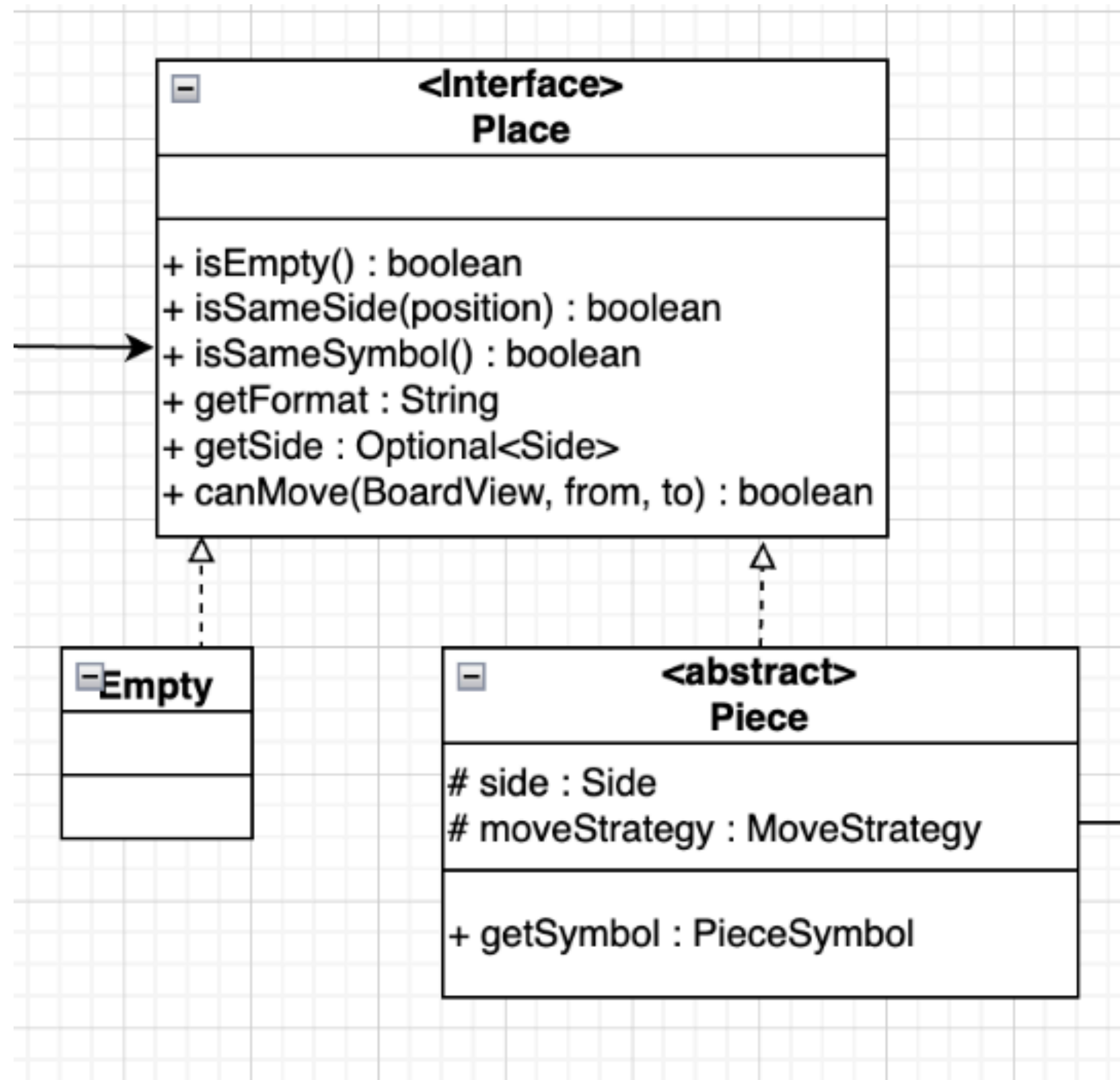
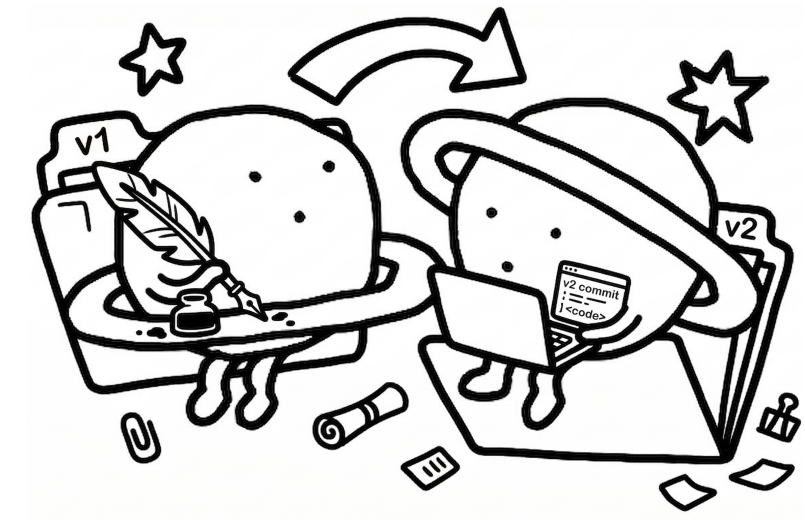
면접관

좋아, 그럼 Null Object 패턴으로 인정하자.

근데 Null Object 패턴의 단점이 뭔지 알아?

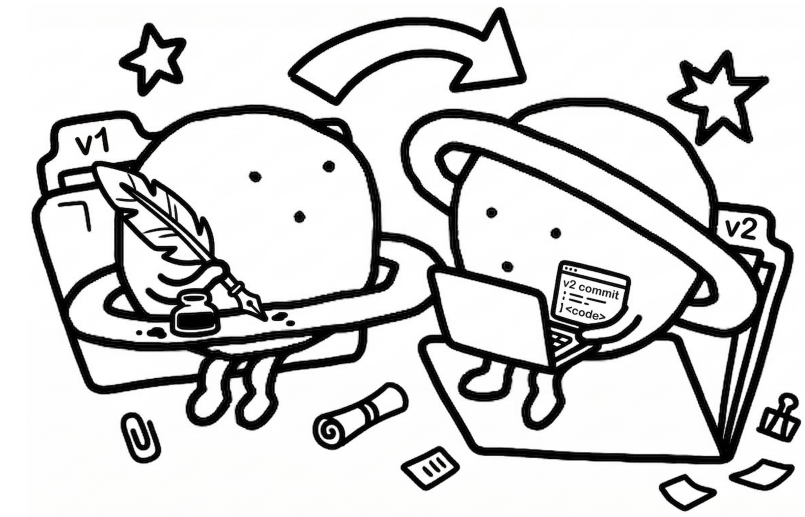
오후 02:50

Null Object Pattern?



Piece != Empty

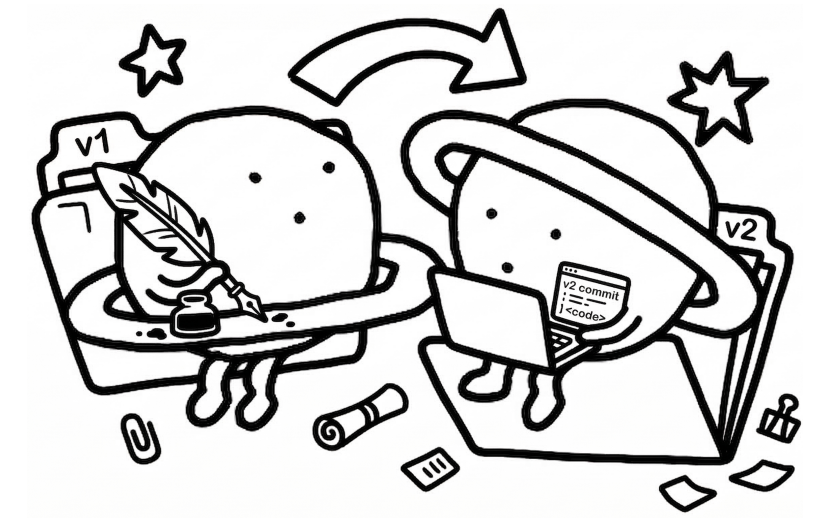
Null Object Pattern



```
public class Empty implements Place {  
  
    @Override  
    public boolean isEmpty() {  
        return true;  
    }  
  
    @Override  
    public Optional<Side> getSide() {  
        return Optional.empty();  
    }  
  
    ...  
  
    @Override  
    public boolean canMove(List<Place> path) {  
        return false;  
    }  
}
```

객체지향적으로 말하면 Empty를
하나의 객체로 모델링하는 패턴이다.

Null Object Pattern

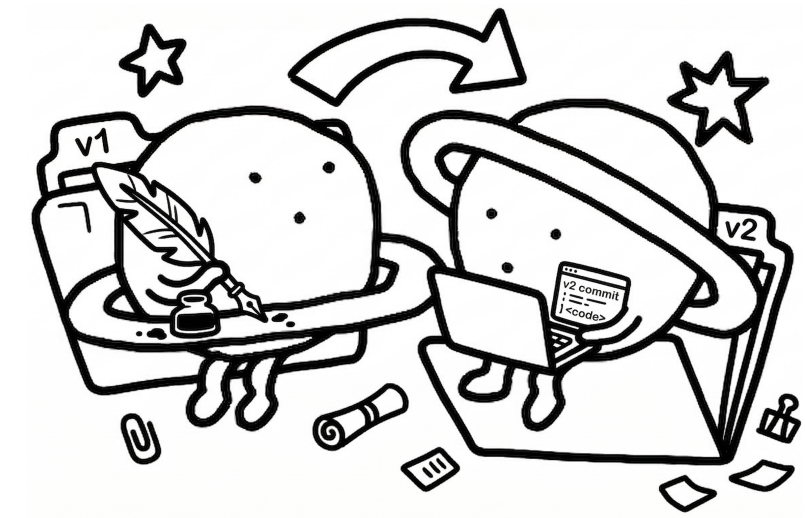


```
public class piece implements Place {  
  
    @Override  
    public boolean isEmpty() {  
        return false;  
    }  
  
    @Override  
    public boolean canMove(List<Position> path ....) {  
        return moveStrategy.canMove(...);  
    }  
}
```

```
// Null Object : 빈 칸  
public class Empty implements Place {  
  
    @Override  
    public boolean isEmpty() {  
        return true;  
    }  
  
    @Override  
    public boolean canMove(List<Position> path ....) {  
        return false;  
    }  
}
```

존재는 하지만 동작은 하지 않는 클래스를 정의한다.

Null Object Pattern



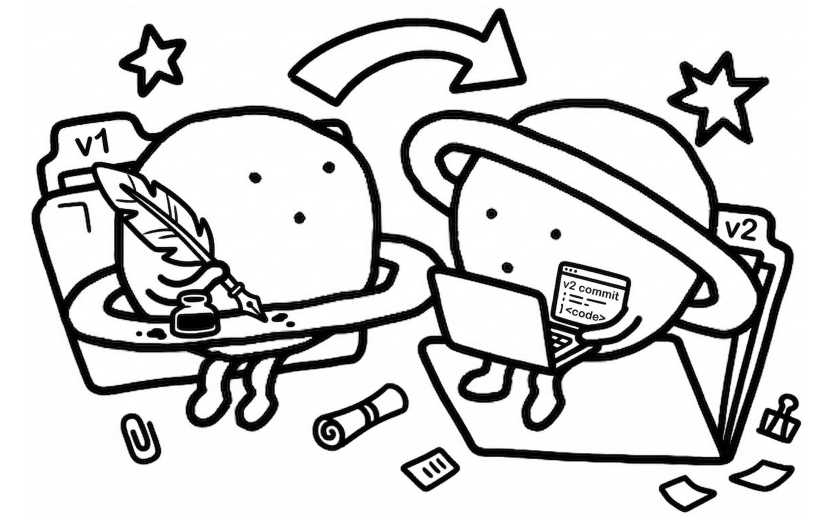
```
Payment payment = paymentService.findPayment(userId);  
  
if (payment != null) {  
    payment.pay(amount);  
}
```



```
Payment payment = paymentService.findPayment(userId);  
payment.pay(amount);
```

-> Null 체크 제거
-> 다형성 클래스가 null을 표시

Null Object Pattern 장점

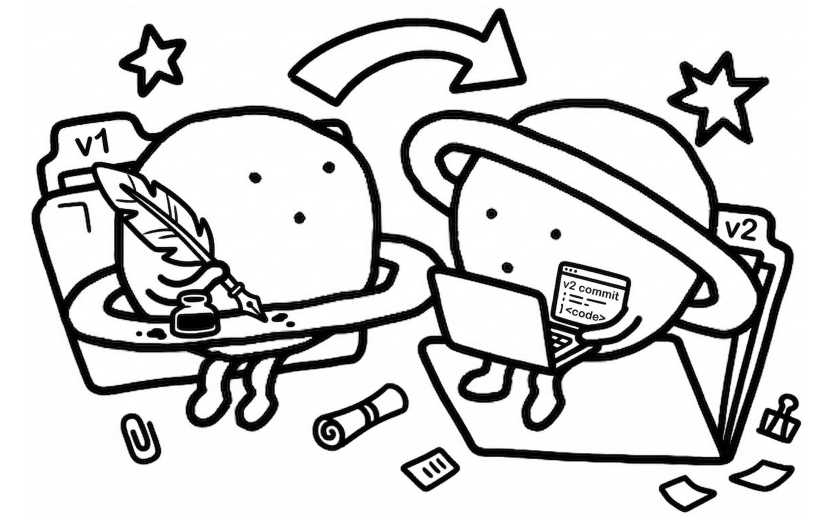


안전함 : Null-safety 구현 가능

코드 간결화 : 지저분한 중복 null 체크 방어 코드가 사라진다.

다형성 활용 : 코드는 이 객체가 진짜인지 널 객체인지 알 필요 없이 일관된 로직 작성 가능

Null Object Pattern 주의점

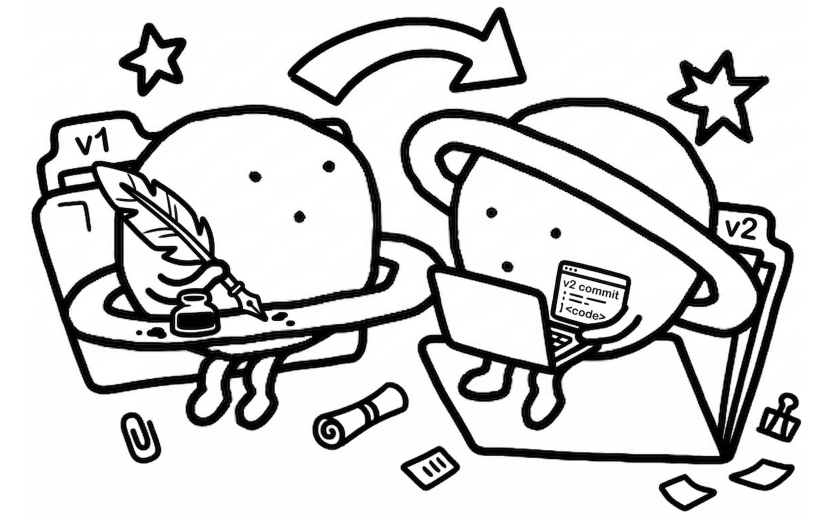


버그 은닉

```
Payment payment = paymentService.findPayment(userId);  
payment.pay(amount);
```

NPE 예외가 사라진 것이 아니라 숨겨진 것일 수도 있다.

Null Object Pattern 주의점

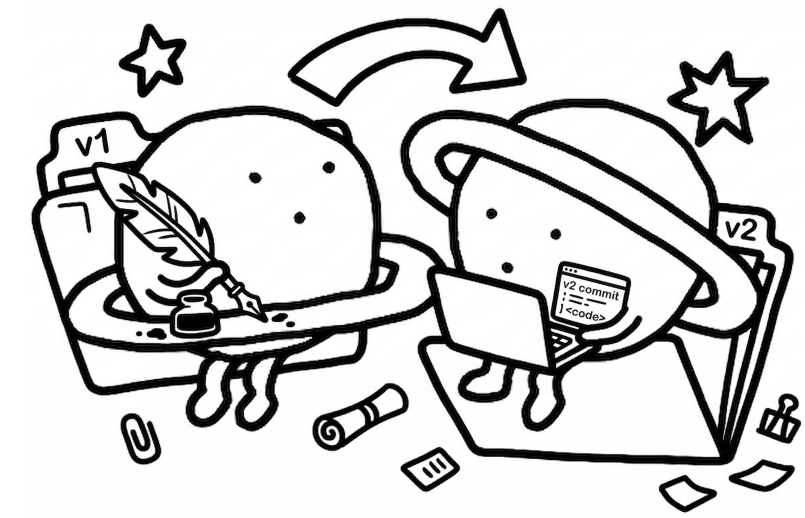


의미 왜곡

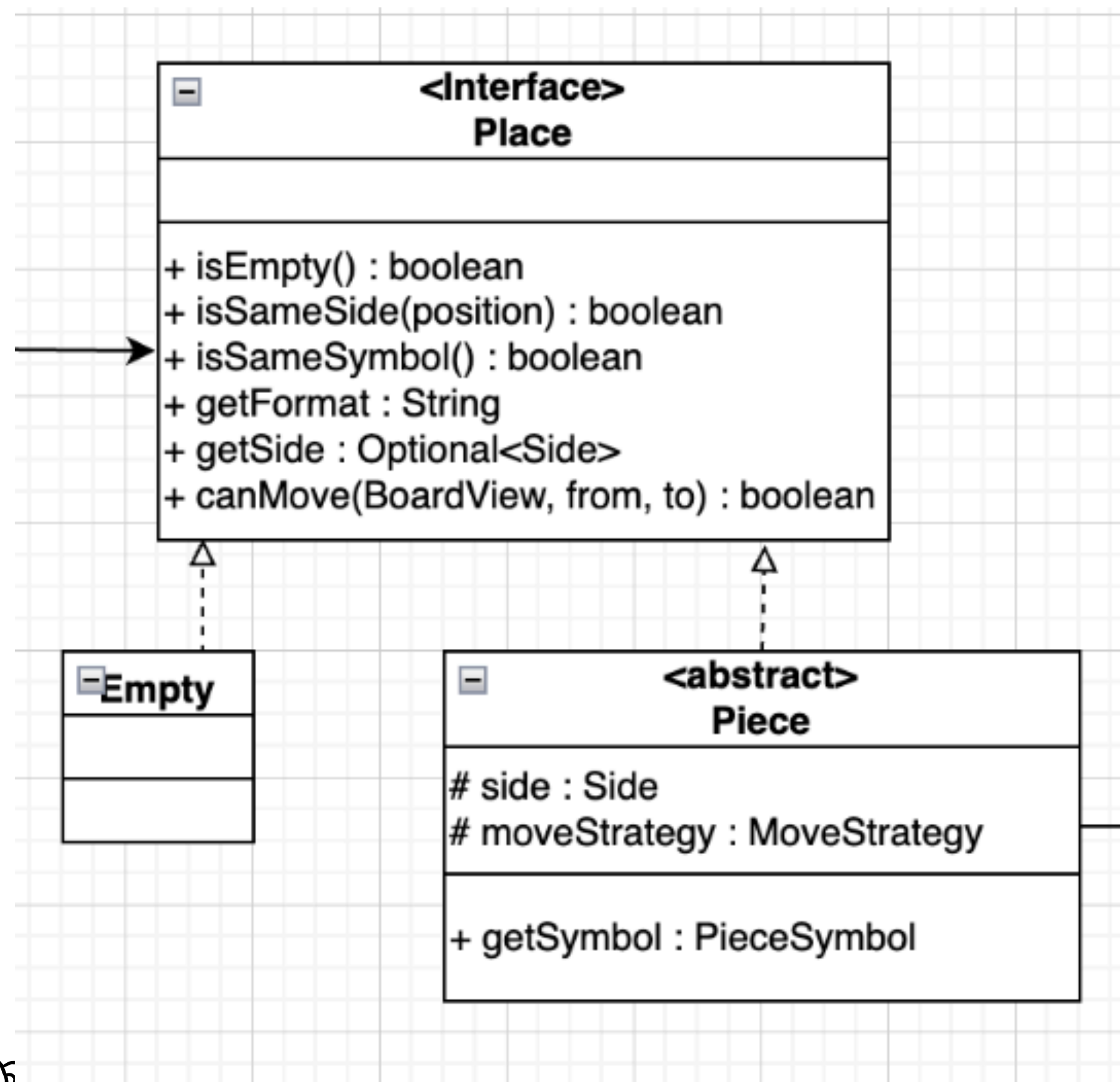
```
Payment payment = paymentService.findPayment(userId);  
payment.pay(amount);
```

'없음'이라는 상태는 자연스럽게 가질 수 있는 개념이 아니다.
"존재하지 않음" 자체가 비즈니스적으로 의미가 있는가?

Null Object Pattern 주의점



변경 전파

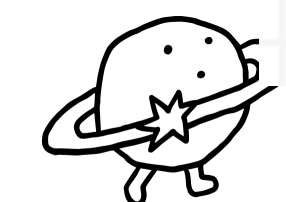


로직과 관련된 메소드를 모두 구현해야한다.

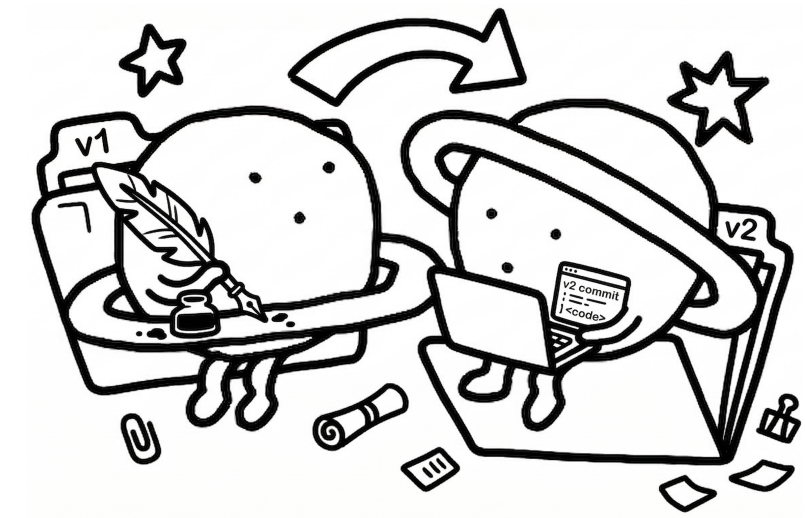
› 의미 없는 기본 구현 증가

› 유지보수 비용 증가

› 실수 가능성 증가



그럼 뭘 써야하지?



if (obj != null) -> 단순할 경우

Optional -> null 가능성이 있고 안전한 처리가 필요한 경우

Null object -> null 자체를 객체로 모델링하기 좋은 환경인 경우

